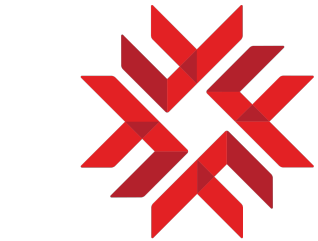
Natural Language Processing 2

Hugging Face for Text Summarization

Fatemeh Dehrouyeh



FANSHAWE

Contents

1. Hugging Face
   - Local Version
   - Cloud Version
2. Accessing Models in Hugging Face
3. Text Summarization
4. Evaluation
   - Pretrained Model
   - Finetuned Model
5. User Interface

Current Section

1 Hugging Face
- Local Version
- Cloud Version

2 Accessing Models in Hugging Face

3 Text Summarization

4 Evaluation

5 User Interface

Why Ollama Can Slow the Workflow

Working with Ollama can make the process slow because:

- Models run locally, so speed depends on your CPU, GPU, RAM, and VRAM.
- Large models require heavy memory and compute.
- Quantized models are faster, but may reduce accuracy.
- Loading and switching models can add extra delay.

A Better Approach: Hugging Face

A better approach:

- Access the model architecture and weights directly.
- Use Hugging Face as a strong source for open model files, configs, and checkpoints.
- Run the model locally when you need control, privacy, or customization.
- Use cloud inference when you need faster setup and scalable compute.

Key point: Hugging Face is a model hub. The model becomes local only when you download and run it locally.

Hugging Face: Local or Cloud Based?

Hugging Face can support both local and cloud-based workflows.

- **Local use:** Download the model weights and architecture from Hugging Face, then run the model on your own machine.
- **Cloud use:** Use Hugging Face APIs, Inference Endpoints, or hosted Spaces to run models on cloud infrastructure.

Key distinction:

- Hugging Face itself is a model hub.
- The model becomes local only when you download it and run it locally.
- Otherwise, inference can happen in the cloud through Hugging Face services.

Current Subsection

1. Hugging Face
   - Local Version
   - Cloud Version
2. Accessing Models in Hugging Face
3. Text Summarization
4. Evaluation
5. User Interface

Code: Local Version

```python
# Install first:
# pip install transformers torch accelerate

from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

# You can replace this with another Hugging Face model ID
MODEL_ID = "Qwen/Qwen2.5-0.5B-Instruct"

# 1. Load tokenizer from Hugging Face
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
# AutoTokenizer automatically loads the tokenizer that belongs to the model.

# 2. Load model architecture + weights from Hugging Face
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID,
    torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32,
    device_map="auto"
)

# 3. Create prompt
prompt = "Explain why accessing model weights directly can be better than using Ollama."

messages = [
    {"role": "system", "content": "You are a clear and concise AI assistant."},
    {"role": "user", "content": prompt}
]
```

Code: Local Version—Continued

```
# 4. Format prompt for instruction-tuned model
text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True
)

# 5. Convert text to model inputs
inputs = tokenizer(text, return_tensors="pt").to(model.device) # pt means PyTorch

# 6. Generate response locally
with torch.no_grad():
    output = model.generate(
        **inputs, # **inputs means "unpack this dictionary into named arguments"
        max_new_tokens=200,
        temperature=0.7,
        do_sample=True # do_sample=True means the model chooses the next token probabilistically, not always by the highest score.
    )

# 7. Decode response
response = tokenizer.decode(output[0], skip_special_tokens=True)

print(response)
```

The code uses comes directly from Hugging Face.

Result

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json: 100% 659/659 [00:00<00:00, 12.3kB/s]
tokenizer_config.json: 7.30k/? [00:00<00:00, 140kB/s]
vocab.json: 2.78M/? [00:00<00:00, 12.8MB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
merges.txt: 1.67M/? [00:00<00:00, 12.4MB/s]
tokenizer.json: 7.03M/? [00:00<00:00, 35.2MB/s]
`torch_dtype` is deprecated! Use `dtype` instead!
model.safetensors: 100% 988M/988M [00:05<00:00, 251MB/s]
Loading weights: 100% 290/290 [00:02<00:00, 121.42it/s, Materializing param=model.norm.weight]
generation_config.json: 100% 242/242 [00:00<00:00, 8.98kB/s]
system
You are a clear and concise AI assistant.
user
Explain why accessing model weights directly can be better than using Ollama.
assistant
Accessing model weights directly is generally considered to be more efficient and faster compared to using Ollama for this purpose. Here's why:

1. **Speed**: Direct access to model weights allows you to quickly retrieve the current state of the network, which is crucial in training and inference stages.

2. **Reduced Memory Usage**: When working with large models or deep learning networks that use many layers, direct access reduces memory usage by allowing you to store only the neces

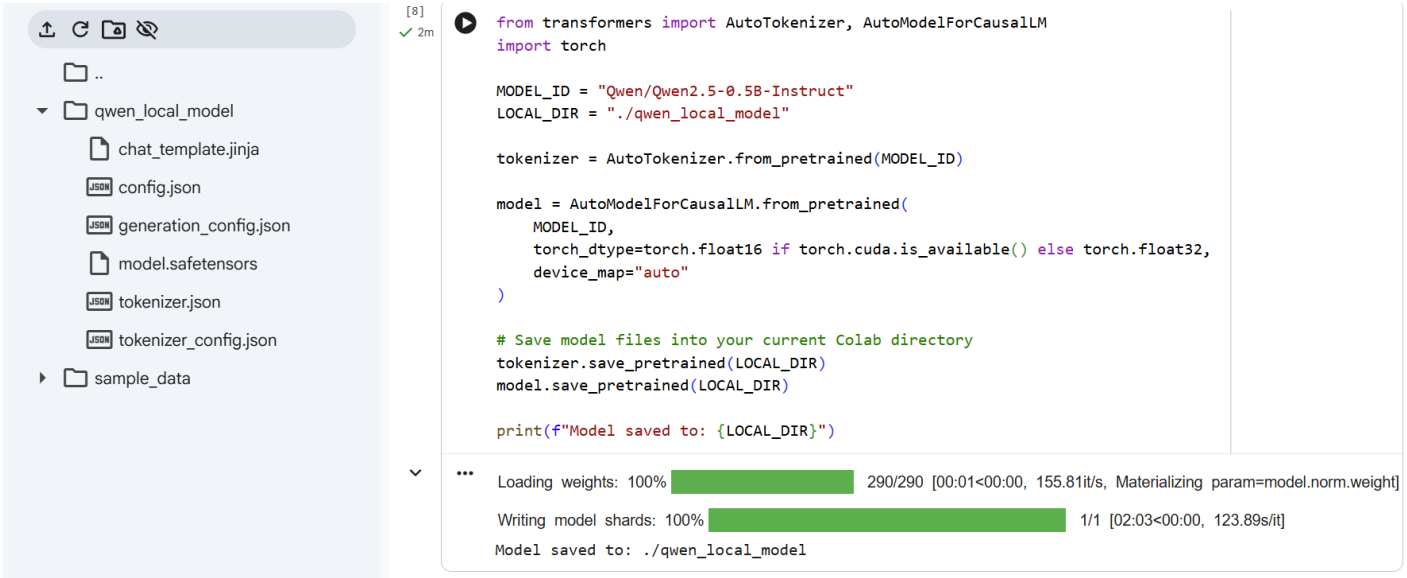
3. **Efficiency in Training**: By fetching the weights immediately after each epoch during training, you avoid the overhead of retraining the same set of parameters repeatedly. This r

4. **Optimization**: In training loops, directly accessing weights can help optimize the process by reducing unnecessary computations and speeding up convergence. For example, if you
```

Explanation

The loaded model is Qwen (also known as Tongyi Qianwen) is a family of highly advanced AI models developed by Alibaba Cloud. It includes general-purpose language models, multimodal models (which can process text, images, and audio), and specialized coding and agentic models.

Hugging Face downloads the model files into Colab's cache directory. To save the model in the current directory:



Current Subsection

1. Hugging Face
   - Local Version
   - Cloud Version
2. Accessing Models in Hugging Face
3. Text Summarization
4. Evaluation
5. User Interface

Why Cloud Inference Needs an API Key

Hugging Face can run models in two ways:

- **Local model:** You download the model weights and run the model on your own machine.
- **Cloud model:** Hugging Face runs the model on its servers through an API.

The cloud version needs an API key because:

- It identifies who is making the request.
- It controls usage limits and billing.
- It protects private, gated, or licensed models.
- It prevents anonymous abuse of Hugging Face compute resources.

Key idea:

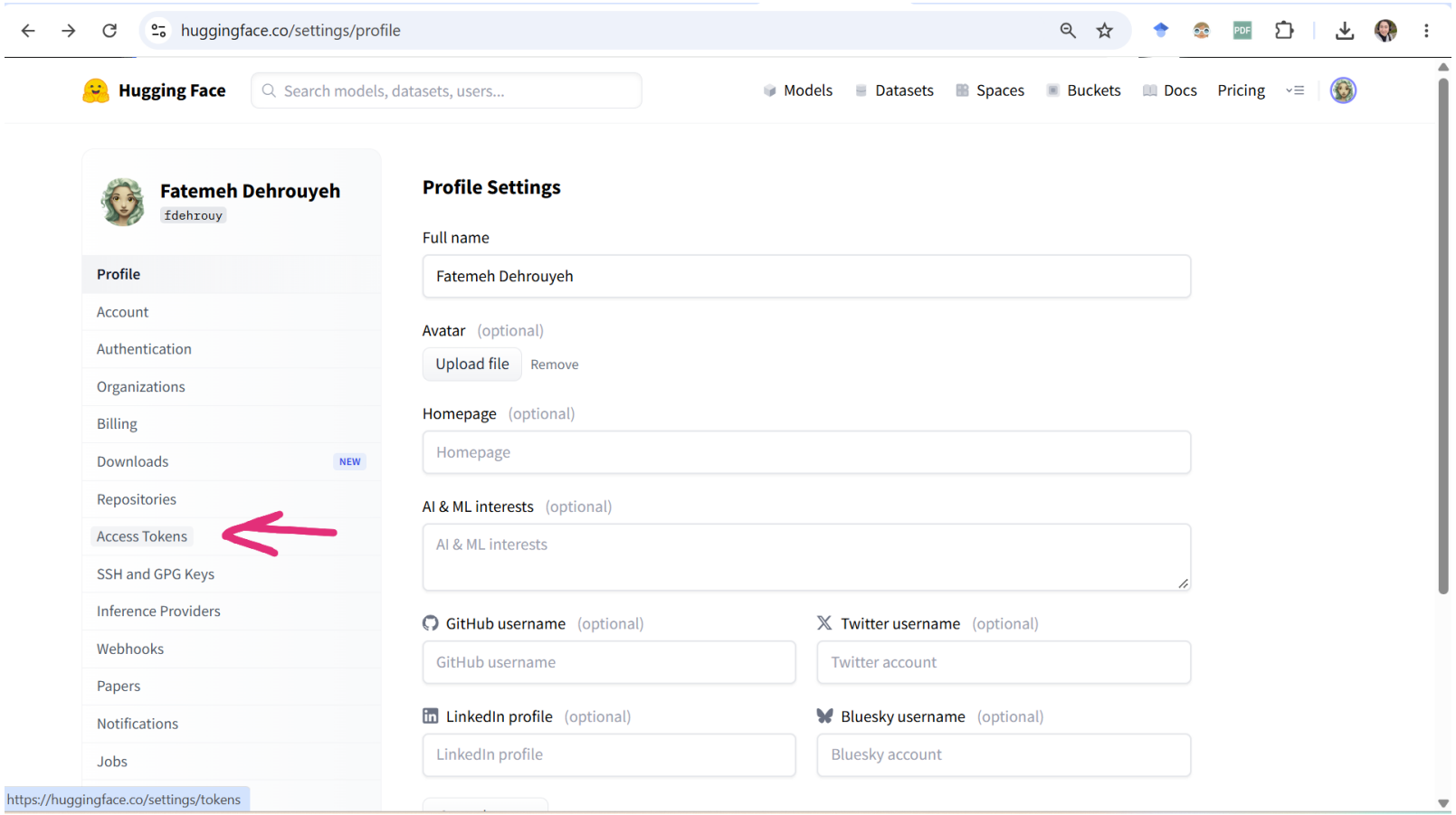
Local inference uses your hardware. Cloud inference uses Hugging Face hardware, so access must be authenticated.

How to Get a Hugging Face API Key

Follow these steps to get a Hugging Face API key, called a User Access Token.

1. **Create or log in to your Hugging Face account**
   Go to Hugging Face (https://huggingface.co/) and sign in.
2. **Open your token settings**
   Go to:
   `Profile picture` → `Settings` → `Access Tokens`
   Access tokens are created from `Settings` → `Access Tokens` → `New token`.
3. **Create a new token**
   Click: `New token`
4. **Choose the token type**
   For cloud inference, choose a token with inference permissions.
   Hugging Face says Inference Providers need a Hugging Face token with inference permissions.
   Use a name like: `colab-inference-token`
   Token type: `Fine-grained`.
   This is recommended because it lets you give the token only the permissions it needs, instead of broad account access.
   Under **Inference**, select: `Make calls to Inference Providers`
5. **Copy the token**
   After creating it, copy it immediately.
   It will look like: `hf_xxxxxxxxxxxxxxxxxxxxxxxxxx`
   Do not share it publicly.

Access Token



Code: Cloud Version

```python
# pip install openai

import os
from openai import OpenAI
from getpass import getpass

hf_token = getpass("Enter your new Hugging Face token: ")

client = OpenAI(
    base_url="https://router.huggingface.co/v1",
    api_key=hf_token,
)

completion = client.chat.completions.create(
    model="katanemo/Arch-Router-1.5B:hf-inference",
    messages=[
        {
            "role": "user",
            "content": "Explain Hugging Face Inference Providers in simple terms."
        }
    ],
    max_tokens=200,
)

print(completion.choices[0].message.content)
```

The code comes from Hugging Face's API integration example. You are installing the openai Python library because Hugging Face supports an API format that is compatible with OpenAI's API.

Result

```
•••  Enter your new Hugging Face token: ··········
     Sure! Hugging Face's Inference Providers refer to the different ways that you can use their pre-tra

     Here’s a simple breakdown:

     1. **Inference Provider**: This is essentially an interface or API that allows your application to

     2. **Model Server**: This refers to the backend infrastructure where the actual computation happens

     3. **API Gateway**: Some providers may have an additional layer of abstraction called an API gatewa
```

Generation vs. Completion

Text generation is the general task of producing new text from an input.

Text completion is a specific type of generation where the model continues unfinished text.

- **Generation:** The model creates a response, summary, explanation, or answer.
- **Completion:** The model continues the prompt from where it stops.

Example:

- **Prompt:** `The future of AI is`
- **Completion:** `likely to transform education, healthcare, and business.`
- **Generation:** `AI will help automate tasks, support decision-making, and create new tools.`

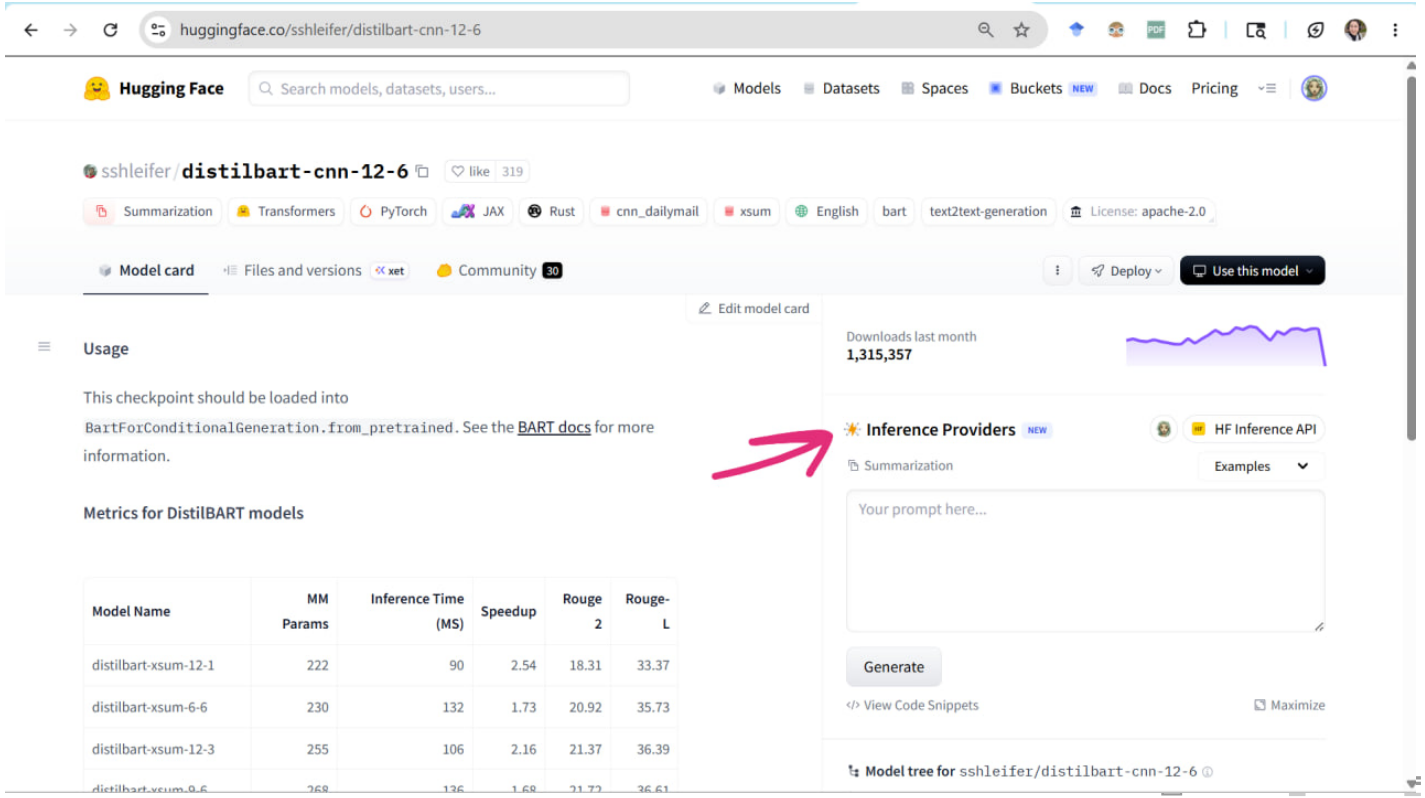
Key idea:

All completion is generation, but not all generation is completion.

Models with Hugging Face Inference Providers

To use a model as a cloud-based service, the model must be available through Hugging Face Inference Providers. We need to check whether the model page shows an `Inference Providers` panel.

If this panel is available, we can send input text to the model using an API token and receive the generated output without downloading the model locally.



Current Section

1 Hugging Face

2 Accessing Models in Hugging Face

3 Text Summarization

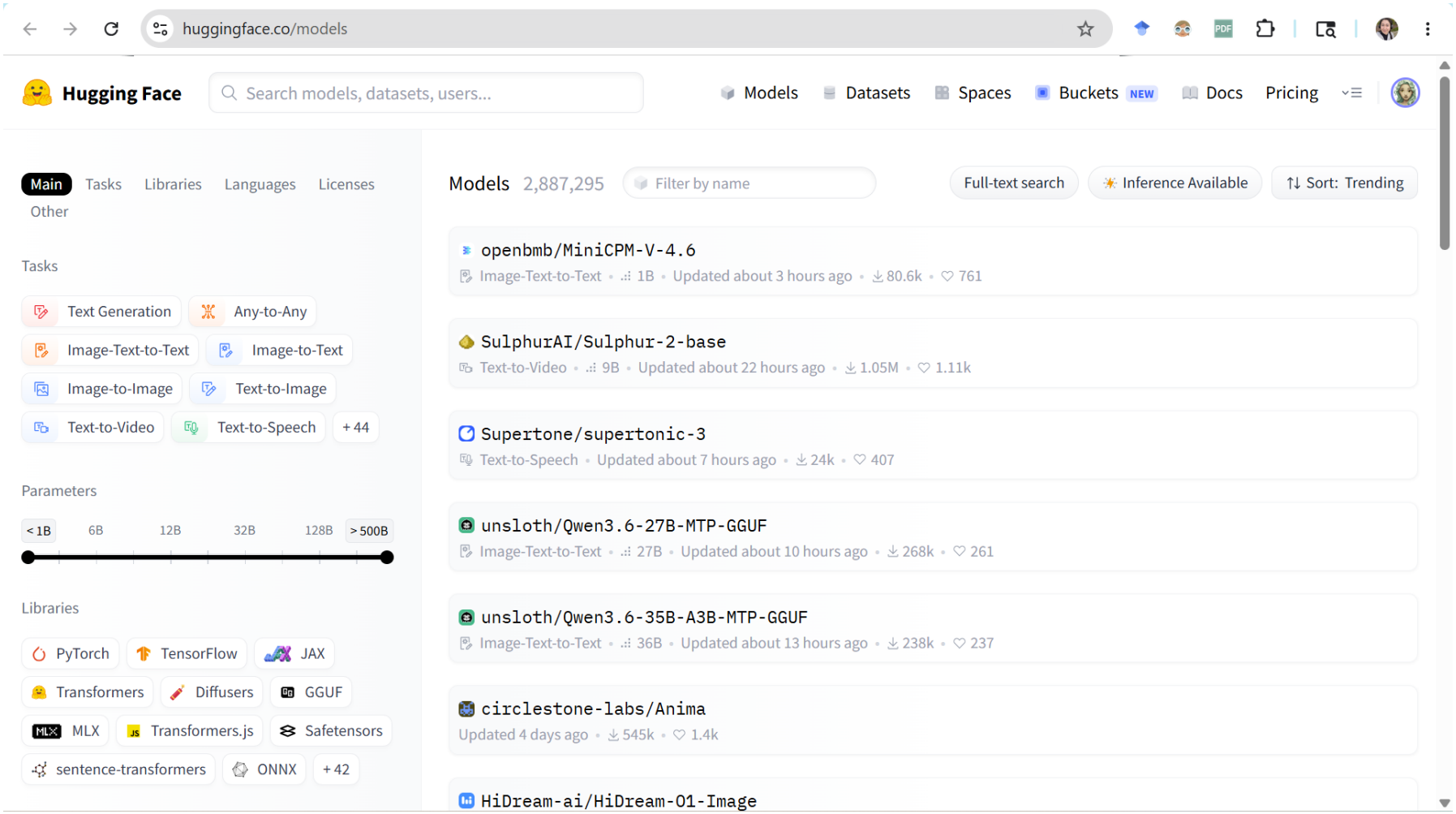
4 Evaluation

5 User Interface

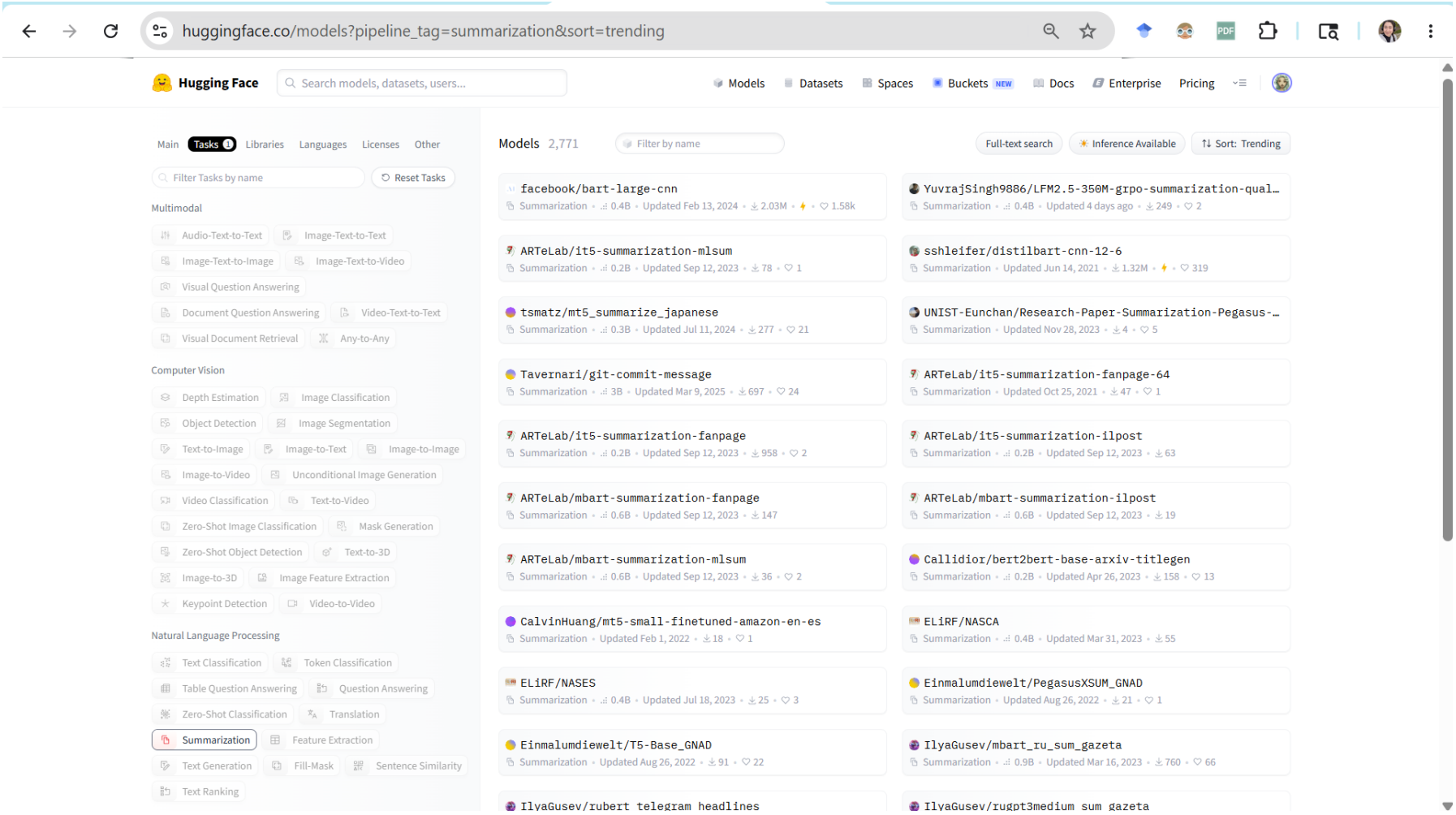
How to Choose a Model on Hugging Face

1. Define the task: summarization, classification, translation, question answering, or generation.
2. Go to the Hugging Face Models page (https://huggingface.co/models).
3. Filter by task, library, language, license, model size, and inference provider.
4. For summarization, choose: `Task` $\rightarrow$ `Natural Language Processing` $\rightarrow$ `Summarization`.
5. Read the model card to check intended use, training data, limitations, and examples.
6. Check model size: smaller models are easier to run locally; larger models may need more RAM or GPU.
7. Test 2–4 models using the same input and compare quality, speed, memory use, and output length.

Hugging Face Models

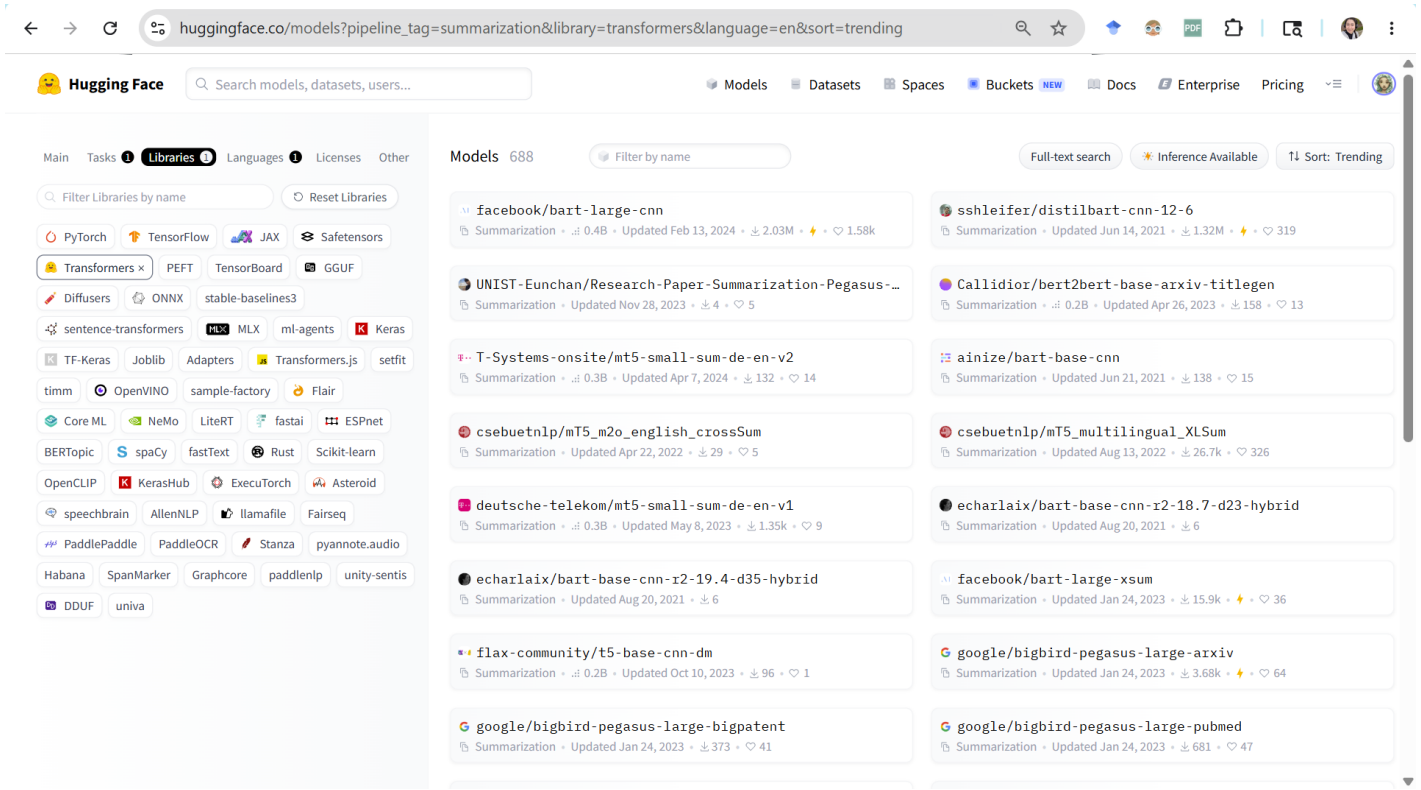


Filtering Summarization



Available Models

After filtering by the Summarization task, English language, and Transformer library, we have the following list of available models:



Choosing a Summarization Model

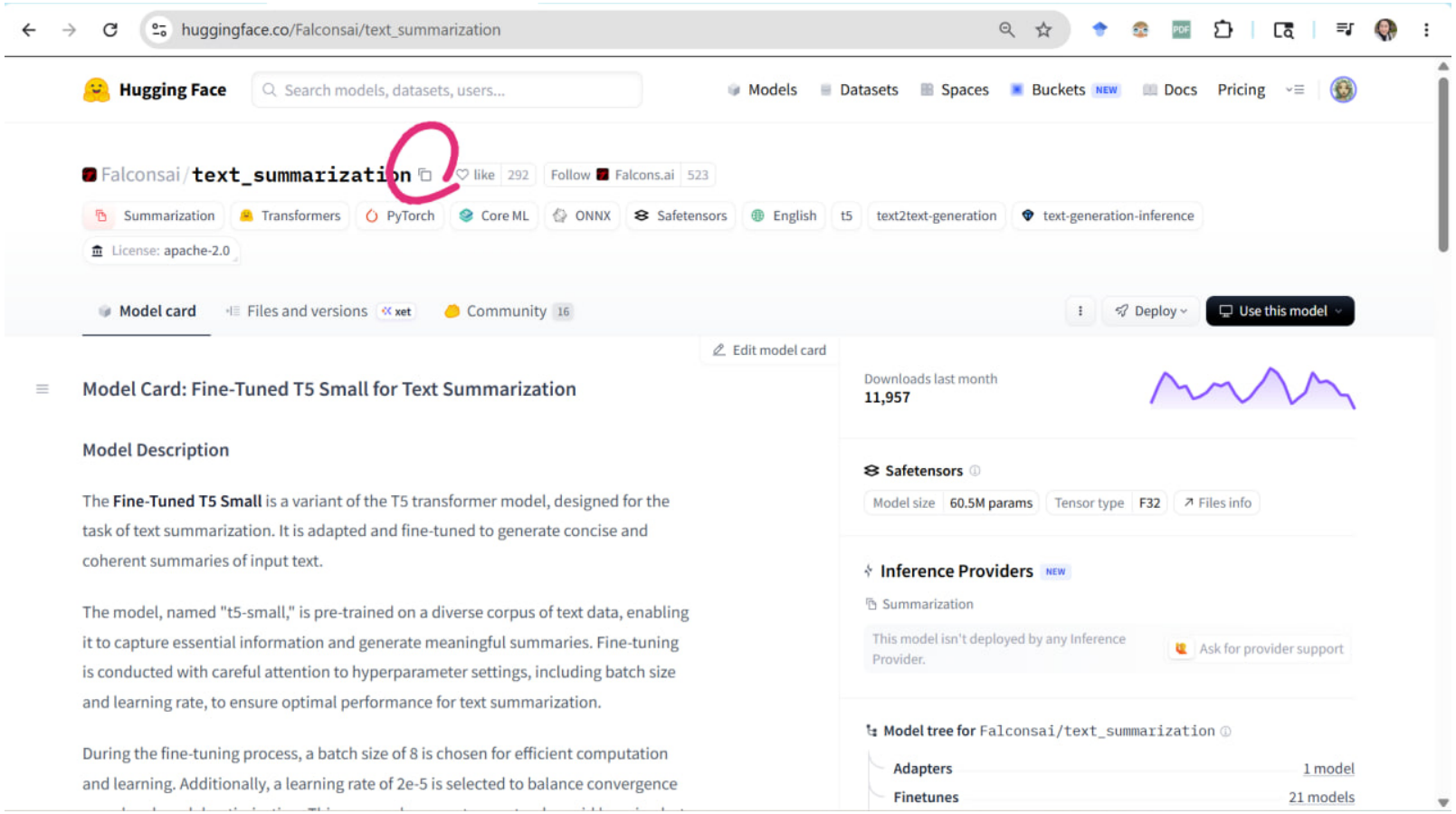
Good starting models:

- `Falconsai/text_summarization`
- `Waris01/google-t5-finetuning-text-summarization`
- `sshleifer/distilbart-cnn-12-6`
- `facebook/bart-large-cnn`
- `google/pegasus-cnn_dailymail`

Simple first choice: `Falconsai/text_summarization`

Selected Model

To access the model, we only need to copy its name:



Current Section

1 Hugging Face

2 Accessing Models in Hugging Face

3 Text Summarization

4 Evaluation

5 User Interface

Testing the Model

```python
# Install if needed
# !pip install -q transformers torch sentencepiece
#-q means quiet

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch

model_name = "Falconsai/text_summarization"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

text = """
Artificial intelligence is changing the way people work, study, and communicate.
In education, AI tools can help students summarize long articles, explain difficult
concepts, and generate practice questions. However, students should still learn how
to evaluate the quality of AI-generated answers because models can make mistakes.
Teachers can use AI as a support tool, but not as a replacement for critical thinking.
"""

input_text = "summarize: " + text

inputs = tokenizer(input_text, return_tensors="pt", max_length=512, truncation=True)

with torch.no_grad():  # Disables gradient calculation because we are only using the model for inference
    summary_ids = model.generate(
        inputs["input_ids"],      # The tokenized input text
        max_length=60,            # Maximum length of the generated summary
        min_length=20,            # Minimum length of the generated summary
        num_beams=4,              # Uses beam search with 4 candidate summaries
        length_penalty=2.0,       # Penalizes very long summaries
        early_stopping=True       # Stops generation when a good complete summary is found
    )

summary = tokenizer.decode(summary_ids[0], skip_special_tokens=True)

print(summary)
```

Result

```
summary
```

```
'AI tools can help students summarize long articles, explain difficult concepts, and generate practice questions. Teachers can use AI as a s
upport tool, but not as a replacement for critical thinking.'
```

As shown in the example, the model is able to generate a proper summary of the input text.
If the goal is general text summarization, the pretrained model may be sufficient. However, if the goal is to adapt the model to a specific domain, writing style, or output format, fine-tuning can help improve its performance for that specific use case.

Fine-Tuning: Local vs. Cloud

Fine-tuning can be performed either locally or in the cloud.

- **Local Fine-Tuning:**
  - Download the pretrained model and its weights.
  - Update the model using a custom training dataset.
  - Requires sufficient local computing resources.

- **Cloud Fine-Tuning:**
  - Use a cloud provider's fine-tuning service.
  - Upload a training dataset to customize the model.
  - The provider manages the training and model weights.
  - Direct access to the weights is not required.

Important: Not all cloud-hosted models support fine-tuning. This depends on the model and the provider.

Next Step

- Choose a suitable summarization dataset.
- Evaluate the pretrained model on the dataset.
- Fine-tune the model using the dataset.
- Compare the evaluation metrics before and after fine-tuning.

Current Section

1 Hugging Face

2 Accessing Models in Hugging Face

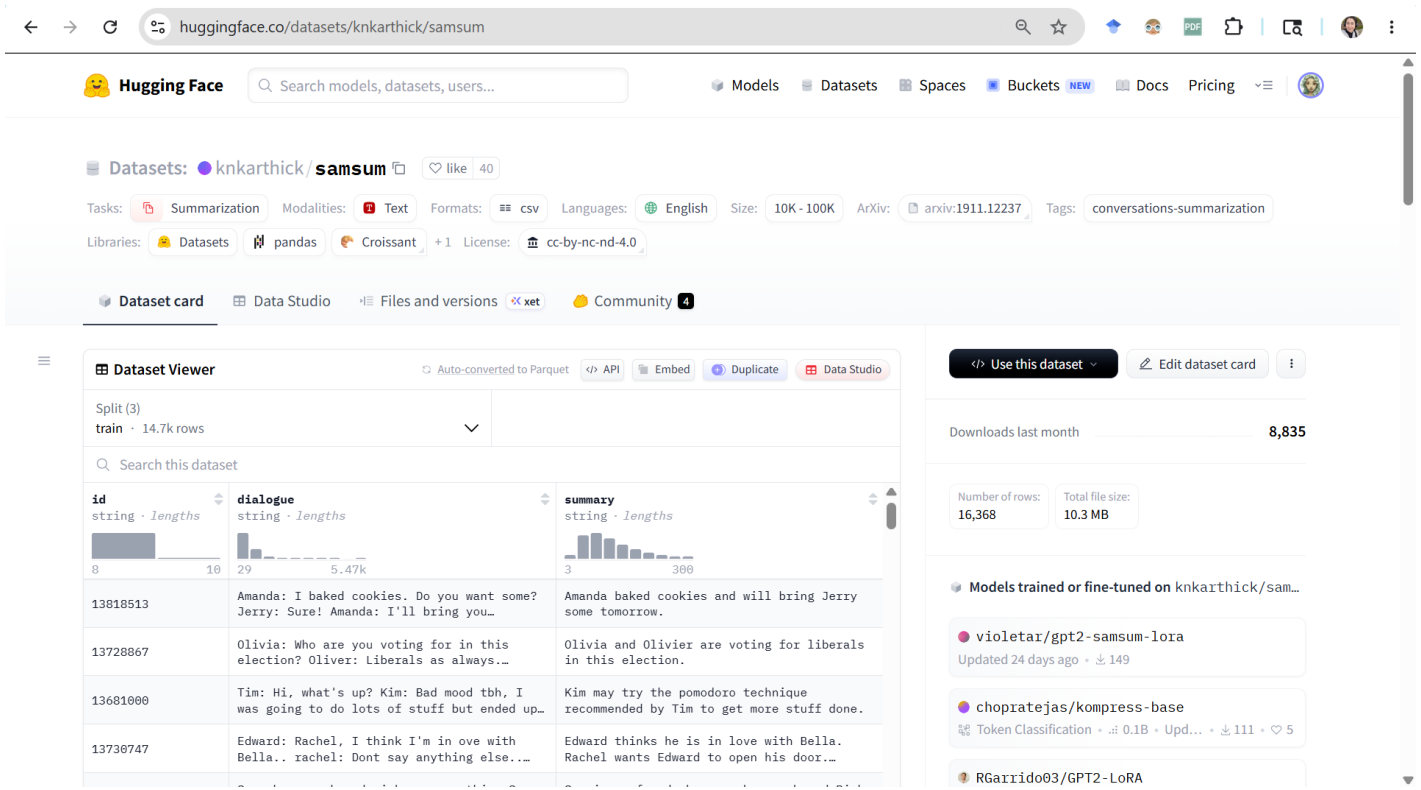
3 Text Summarization

4 Evaluation
- Pretrained Model
- Finetuned Model

5 User Interface

Dataset Selection

We select the `knkarthick/samsum` dataset from Hugging Face. This dataset contains dialogue examples paired with human-written summaries. It is suitable because it is relatively small, easy to understand, and directly related to **abstractive text summarization**.



Dataset Link: https://huggingface.co/datasets/knkarthick/samsum

Loading Dataset

```
# !pip install datasets

from datasets import load_dataset

dataset = load_dataset("knkarthick/samsum")

print(dataset)
print(dataset["train"][0])
```

```
README.md: 4.36k/? [00:00<00:00, 182kB/s]
train.csv: 9.26M/? [00:00<00:00, 58.5MB/s]
validation.csv: 504k/? [00:00<00:00, 15.5MB/s]
test.csv: 522k/? [00:00<00:00, 28.2MB/s]
Generating train split: 100% 14731/14731 [00:00<00:00, 33105.54 examples/s]
Generating validation split: 100% 818/818 [00:00<00:00, 14316.94 examples/s]
Generating test split: 100% 819/819 [00:00<00:00, 18938.05 examples/s]
DatasetDict({
    train: Dataset({
        features: ['id', 'dialogue', 'summary'],
        num_rows: 14731
    })
    validation: Dataset({
        features: ['id', 'dialogue', 'summary'],
        num_rows: 818
    })
    test: Dataset({
        features: ['id', 'dialogue', 'summary'],
        num_rows: 819
    })
})
{'id': '13818513', 'dialogue': "Amanda: I baked  cookies. Do you want some?\nJerry: Sure!\nAmanda: I'll bring you tomorrow :-)", 'summary': 'Amanda baked cookies and will bring Jerry
```

Current Subsection

1 Hugging Face

2 Accessing Models in Hugging Face

3 Text Summarization

4 Evaluation
- Pretrained Model
- Finetuned Model

5 User Interface

Load dataset and model

```
!pip install -q transformers torch sentencepiece datasets evaluate rouge_score sacrebleu nltk bert_score

from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch
from tqdm import tqdm

dataset = load_dataset("knkarthick/samsum")

model_name = "Falconsai/text_summarization"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

device = "cuda" if torch.cuda.is_available() else "cpu"
model = model.to(device)

print(dataset)
print("Device:", device)
```

Using Test Set and Generating Summaries

```
test_data = dataset["test"].select(range(100)) #using a small subset
```

```
def generate_summary(text):
    input_text = "summarize: " + text

    inputs = tokenizer(
        input_text,
        return_tensors="pt",
        max_length=512,
        truncation=True
    ).to(device)

    with torch.no_grad():
        summary_ids = model.generate(
            inputs["input_ids"],
            max_length=80,
            min_length=10,
            num_beams=4,
            length_penalty=2.0,
            early_stopping=True
        )

    summary = tokenizer.decode(
        summary_ids[0],
        skip_special_tokens=True
    )

    return summary
```

Run prediction on the test set

```
predictions = []
references = []

for example in tqdm(test_data):
    dialogue = example["dialogue"]
    reference_summary = example["summary"]

    generated_summary = generate_summary(dialogue)

    predictions.append(generated_summary)
    references.append(reference_summary)

print("Example prediction:")
print(predictions[0])

print("\nReference summary:")
print(references[0])
```

```
100%|██████████| 100/100 [05:52<00:00,  3.53s/it]Example prediction:
Amanda: Lemme check Hannah: file_gif> Amanda: Sorry, can't find it. Amanda: Ask Larry

Reference summary:
Hannah needs Betty's number but Amanda doesn't have it. She needs to contact Larry.
```

Evaluate with ROUGE, BLEU, METEOR, and BERTScore

```
import evaluate
import numpy as np

rouge = evaluate.load("rouge")
bleu = evaluate.load("bleu")
meteor = evaluate.load("meteor")
bertscore = evaluate.load("bertscore")

rouge_results = rouge.compute(predictions=predictions, references=references)

bleu_results = bleu.compute(predictions=predictions, references=[[ref] for ref in references])

meteor_results = meteor.compute(predictions=predictions, references=references)

bertscore_results = bertscore.compute(predictions=predictions, references=references, lang="en")

results = {
    "rouge1": rouge_results["rouge1"],
    "rouge2": rouge_results["rouge2"],
    "rougeL": rouge_results["rougeL"],
    "rougeLsum": rouge_results["rougeLsum"],
    "bleu": bleu_results["bleu"],
    "meteor": meteor_results["meteor"],
    "bertscore_precision": np.mean(bertscore_results["precision"]),
    "bertscore_recall": np.mean(bertscore_results["recall"]),
    "bertscore_f1": np.mean(bertscore_results["f1"])
}

results
```

Display results clearly

```
for metric, value in results.items():
    print(f"{metric}: {value:.4f}")
```

```
rouge1: 0.2457
rouge2: 0.0632
rougeL: 0.1931
rougeLsum: 0.1934
bleu: 0.0330
meteor: 0.2481
bertscore_precision: 0.8318
bertscore_recall: 0.8843
bertscore_f1: 0.8570
```

Evaluation of the Pretrained Model

- We first evaluated the pretrained model on the test set of the `SAMSum` dataset.
- The generated summaries were compared with the human-written reference summaries.
- We used automatic evaluation metrics, including ROUGE, BLEU, METEOR, and BERTScore.
- These results provide the baseline performance before fine-tuning.
- The same test set should be used before and after fine-tuning to ensure a fair comparison.
- The next step is to fine-tune the model and evaluate its performance again.

Current Subsection

1 Hugging Face

2 Accessing Models in Hugging Face

3 Text Summarization

4 Evaluation
- Pretrained Model
- Finetuned Model

5 User Interface

Load dataset and pretrained model

```
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch

dataset = load_dataset("knkarthick/samsum")

model_name = "Falconsai/text_summarization"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

device = "cuda" if torch.cuda.is_available() else "cpu"
model = model.to(device)

print(dataset)
print("Device:", device)
```

Note: This model is based on T5. T5 is an encoder-decoder model, also called a sequence-to-sequence model.

Preprocess the dataset

```python
# Use small subsets
train_data = dataset["train"].select(range(1000))
validation_data = dataset["validation"].select(range(100))
test_data = dataset["test"].select(range(100))
```

```python
# Preprocess the dataset
max_input_length = 512
max_target_length = 80

def preprocess_function(examples):
    inputs = ["summarize: " + dialogue for dialogue in examples["dialogue"]]
    targets = examples["summary"]

    model_inputs = tokenizer(
        inputs,
        max_length=max_input_length,
        truncation=True
    )

    labels = tokenizer(
        text_target=targets,
        max_length=max_target_length,
        truncation=True
    )

    model_inputs["labels"] = labels["input_ids"]

    return model_inputs
```

Output of the Preprocessing Function

The preprocessing function returns a dictionary used for fine-tuning.

The `input_ids` are added automatically by the tokenizer in this line:

```
model_inputs = tokenizer(inputs, max_length=max_input_length, truncation=True)
```

Then, the target summaries are added manually as `labels`:

```
model_inputs["labels"] = labels["input_ids"]
```

At the end, we have something like this:

```
{ "input_ids":  [21603, 10, 1234, 56, ...],
  "attention_mask":  [1, 1, 1, 1, ...],
  "labels":  [789, 45, 982, 17, ...]  }
```

- `input_ids`: tokenized dialogue input.
- `attention_mask`: indicates which tokens are real input tokens and which tokens are padding. A value of `1` means the model should use the token, while a value of `0` means the token should be ignored.
- `labels`: tokenized target summary used for training.

Tokenize train and validation data

```
tokenized_train = train_data.map(
    preprocess_function,
    batched=True,
    remove_columns=train_data.column_names
)

tokenized_validation = validation_data.map(
    preprocess_function,
    batched=True,
    remove_columns=validation_data.column_names
)
```

```
Map: 100% 1000/1000 [00:01<00:00, 715.70 examples/s]
Map: 100% 100/100 [00:00<00:00, 927.67 examples/s]
```

remove_columns=train_data.column_names removes the original text columns after preprocessing. It Keeps only the columns needed for training.

Set up fine-tuning

```python
from transformers import DataCollatorForSeq2Seq
from transformers import Seq2SeqTrainingArguments, Seq2SeqTrainer

# Creates a data collator that prepares batches during training
data_collator = DataCollatorForSeq2Seq(
    tokenizer=tokenizer,  # Uses the tokenizer to pad input_ids and labels correctly
    model=model           # Uses the model structure to prepare decoder inputs when needed
)

# Defines the training settings
training_args = Seq2SeqTrainingArguments(
    output_dir="./finetuned_samsum_model",  # Folder where checkpoints and model files will be saved
    eval_strategy="epoch",                  # Evaluate the model at the end of each epoch
    save_strategy="epoch",                  # Save a checkpoint at the end of each epoch
    learning_rate=2e-5,                     # Step size used by the optimizer during training
    per_device_train_batch_size=4,          # Number of training examples processed at once on each GPU or CPU
    per_device_eval_batch_size=4,           # Number of validation examples processed at once on each GPU or CPU
    weight_decay=0.01,                      # Regularization value to reduce overfitting
    save_total_limit=2,                     # Keep only the latest 2 saved checkpoints
    num_train_epochs=1,                     # Number of full passes through the training dataset
    predict_with_generate=True,             # Generate summaries during evaluation instead of only computing loss
    fp16=torch.cuda.is_available(),         # Use faster 16-bit training if a GPU is available
    logging_steps=50,                       # Print training logs every 50 steps
    report_to="none"                       # Disable reporting to external tools like Weights & Biases
)
```

DataCollatorForSeq2Seq prepares each training batch for a sequence-to-sequence model like T5. weight_decay is a regularization technique. It slightly penalizes large model weights during training.

Fine-tune the model

```
trainer = Seq2SeqTrainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_validation,
    processing_class=tokenizer,
    data_collator=data_collator
)

trainer.train()
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning:
  super().__init__(loader)
```

[250/250 17:02, Epoch 1/1]

Epoch	Training Loss	Validation Loss
1	2.394690	2.101261

Writing model shards: 100% 1/1 [00:06<00:00, 6.60s/it]

```
TrainOutput(global_step=250, training_loss=2.449457305908203, metrics={'train_runtime': 1
0.242, 'total_flos': 69931531763712.0, 'train_loss': 2.449457305908203, 'epoch': 1.0})
```

Save the fine-tuned model

```
fine_tuned_model_path = "./finetuned_samsum_model_final"

trainer.save_model(fine_tuned_model_path)
tokenizer.save_pretrained(fine_tuned_model_path)
```

```
Writing model shards: 100% 1/1 [00:06<00:00, 6.42s/it]
('./finetuned_samsum_model_final/tokenizer_config.json',
 './finetuned_samsum_model_final/tokenizer.json')
```

Load the fine-tuned model for evaluation

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

fine_tuned_model_path = "./finetuned_samsum_model_final"

tokenizer = AutoTokenizer.from_pretrained(fine_tuned_model_path)
model = AutoModelForSeq2SeqLM.from_pretrained(fine_tuned_model_path)

device = "cuda" if torch.cuda.is_available() else "cpu"
model = model.to(device)
```

```
Loading weights: 100% 131/131 [00:00<00:00, 554.45it/s, Materializing param=shared.weight]
```

Use the same summary generation function

```
def generate_summary(text):
    input_text = "summarize: " + text

    inputs = tokenizer(
        input_text,
        return_tensors="pt",
        max_length=512,
        truncation=True
    ).to(device)

    with torch.no_grad():
        summary_ids = model.generate(
            inputs["input_ids"],
            max_length=80,
            min_length=10,
            num_beams=4,
            length_penalty=2.0,
            early_stopping=True
        )

    summary = tokenizer.decode(
        summary_ids[0],
        skip_special_tokens=True
    )

    return summary
```

Generate summaries on the same test set

```python
from tqdm import tqdm

predictions_finetuned = []
references_finetuned = []

for example in tqdm(test_data):
    dialogue = example["dialogue"]
    reference_summary = example["summary"]

    generated_summary = generate_summary(dialogue)

    predictions_finetuned.append(generated_summary)
    references_finetuned.append(reference_summary)

print("Example fine-tuned prediction:")
print(predictions_finetuned[0])

print("\nReference summary:")
print(references_finetuned[0])
```

```
100%|██████████| 100/100 [02:34<00:00,  1.55s/it]Example fine-tuned prediction:
Amanda called her last time we were at the park together Hannah. He called her last time we were at the park together.

Reference summary:
Hannah needs Betty's number but Amanda doesn't have it. She needs to contact Larry.
```

Evaluate the fine-tuned model

```python
import evaluate
import numpy as np

rouge = evaluate.load("rouge")
bleu = evaluate.load("bleu")
meteor = evaluate.load("meteor")
bertscore = evaluate.load("bertscore")

rouge_results_finetuned = rouge.compute(predictions=predictions_finetuned, references=references_finetuned)

bleu_results_finetuned = bleu.compute(predictions=predictions_finetuned, references=[[ref] for ref in references_finetuned])

meteor_results_finetuned = meteor.compute(predictions=predictions_finetuned, references=references_finetuned)

bertscore_results_finetuned = bertscore.compute(predictions=predictions_finetuned, references=references_finetuned, lang="en")

results_finetuned = {
    "rouge1": rouge_results_finetuned["rouge1"],
    "rouge2": rouge_results_finetuned["rouge2"],
    "rougeL": rouge_results_finetuned["rougeL"],
    "rougeLsum": rouge_results_finetuned["rougeLsum"],
    "bleu": bleu_results_finetuned["bleu"],
    "meteor": meteor_results_finetuned["meteor"],
    "bertscore_precision": np.mean(bertscore_results_finetuned["precision"]),
    "bertscore_recall": np.mean(bertscore_results_finetuned["recall"]),
    "bertscore_f1": np.mean(bertscore_results_finetuned["f1"])
}

results_finetuned
```

Print fine-tuned results

```
for metric, value in results_finetuned.items():
    print(f"{metric}: {value:.4f}")
```

```
rouge1: 0.3296
rouge2: 0.1190
rougeL: 0.2699
rougeLsum: 0.2695
bleu: 0.0925
meteor: 0.2973
bertscore_precision: 0.8889
bertscore_recall: 0.8891
bertscore_f1: 0.8888
```

Compare pretrained vs fine-tuned results

```
print("Metric Comparison")
print("-" * 60)

for metric in results.keys():
    pretrained_score = results[metric]
    finetuned_score = results_finetuned[metric]
    difference = finetuned_score - pretrained_score

    print(
        f"{metric}: "
        f"pretrained = {pretrained_score:.4f}, "
        f"fine-tuned = {finetuned_score:.4f}, "
        f"difference = {difference:.4f}"
    )
```

```
Metric Comparison
------------------------------------------------------------
rouge1: pretrained = 0.2457, fine-tuned = 0.3296, difference = 0.0839
rouge2: pretrained = 0.0632, fine-tuned = 0.1190, difference = 0.0558
rougeL: pretrained = 0.1931, fine-tuned = 0.2699, difference = 0.0769
rougeLsum: pretrained = 0.1934, fine-tuned = 0.2695, difference = 0.0761
bleu: pretrained = 0.0330, fine-tuned = 0.0925, difference = 0.0595
meteor: pretrained = 0.2481, fine-tuned = 0.2973, difference = 0.0491
bertscore_precision: pretrained = 0.8318, fine-tuned = 0.8889, difference = 0.0571
bertscore_recall: pretrained = 0.8843, fine-tuned = 0.8891, difference = 0.0048
bertscore_f1: pretrained = 0.8570, fine-tuned = 0.8888, difference = 0.0318
```

Current Section

1 Hugging Face

2 Accessing Models in Hugging Face

3 Text Summarization

4 Evaluation

5 User Interface

Adding a User Interface

For this project, we want to add `Gradio` web interface:

- The user enters a dialogue or paragraph.
- The model generates a summary.
- The output is displayed in the web interface.

Gradio

Continue in Google Colab:

```
!pip install -q gradio

import gradio as gr

def summarize_text(text):
    summary = generate_summary(text)
    return summary

demo = gr.Interface(
    fn=summarize_text,
    inputs=gr.Textbox(
        lines=8,
        placeholder="Enter a dialogue or paragraph here..."
    ),
    outputs=gr.Textbox(label="Generated Summary"),
    title="Text Summarization Demo",
    description="This app summarizes input text using a fine-tuned T5-based model."
)

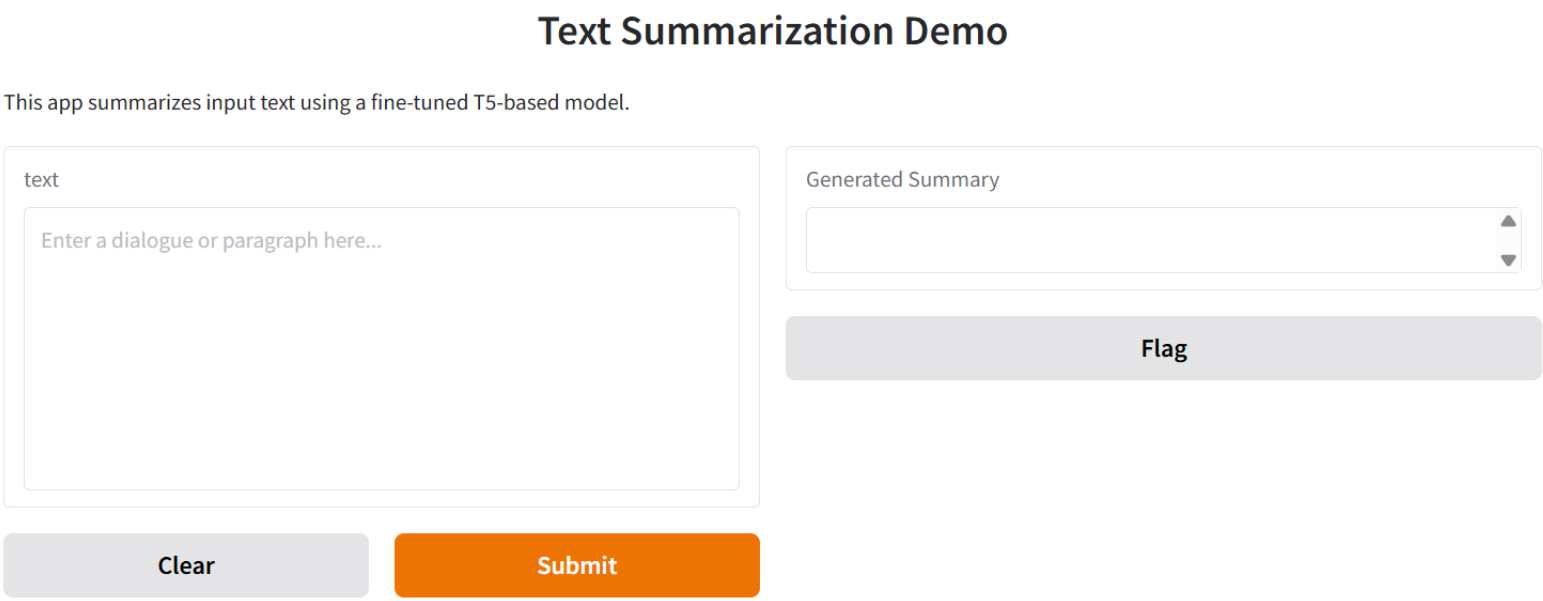
demo.launch(share=True)
```

Result

The result also appears in Google Colab:

```
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://0e01baef8e212bfe89.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working director
```

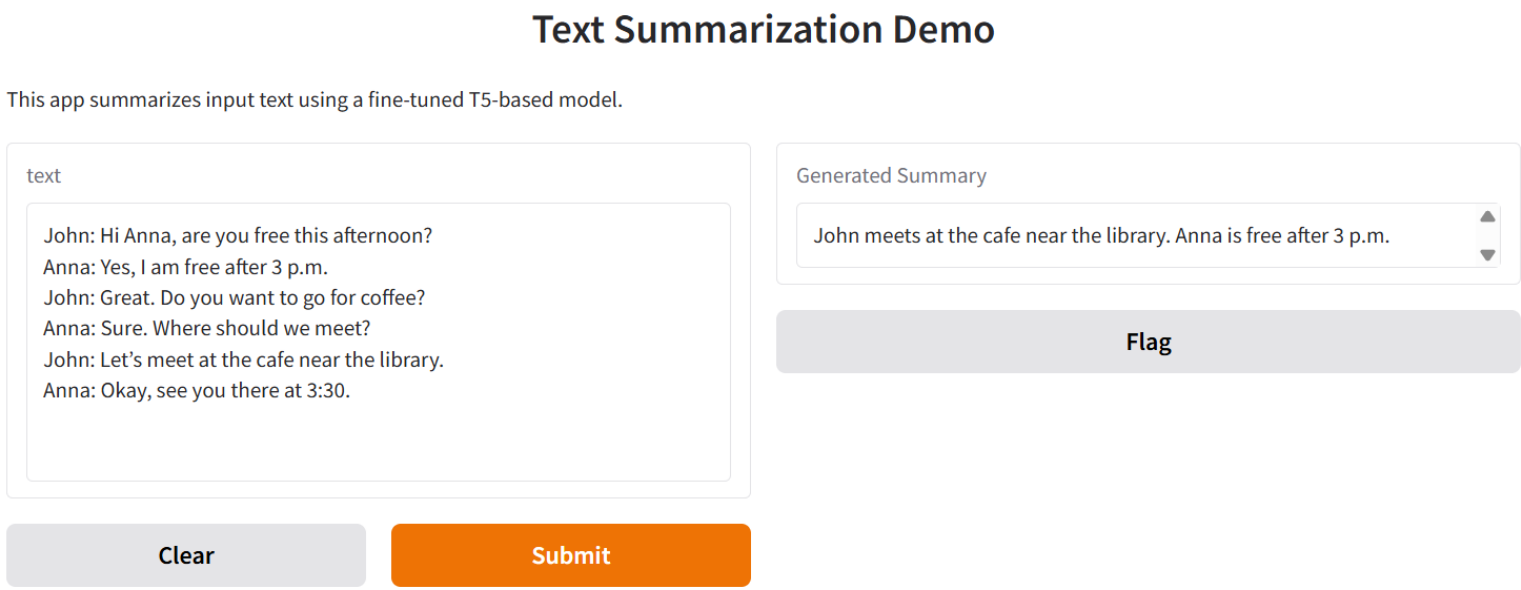


Testing

You can try an example in Google Colab:

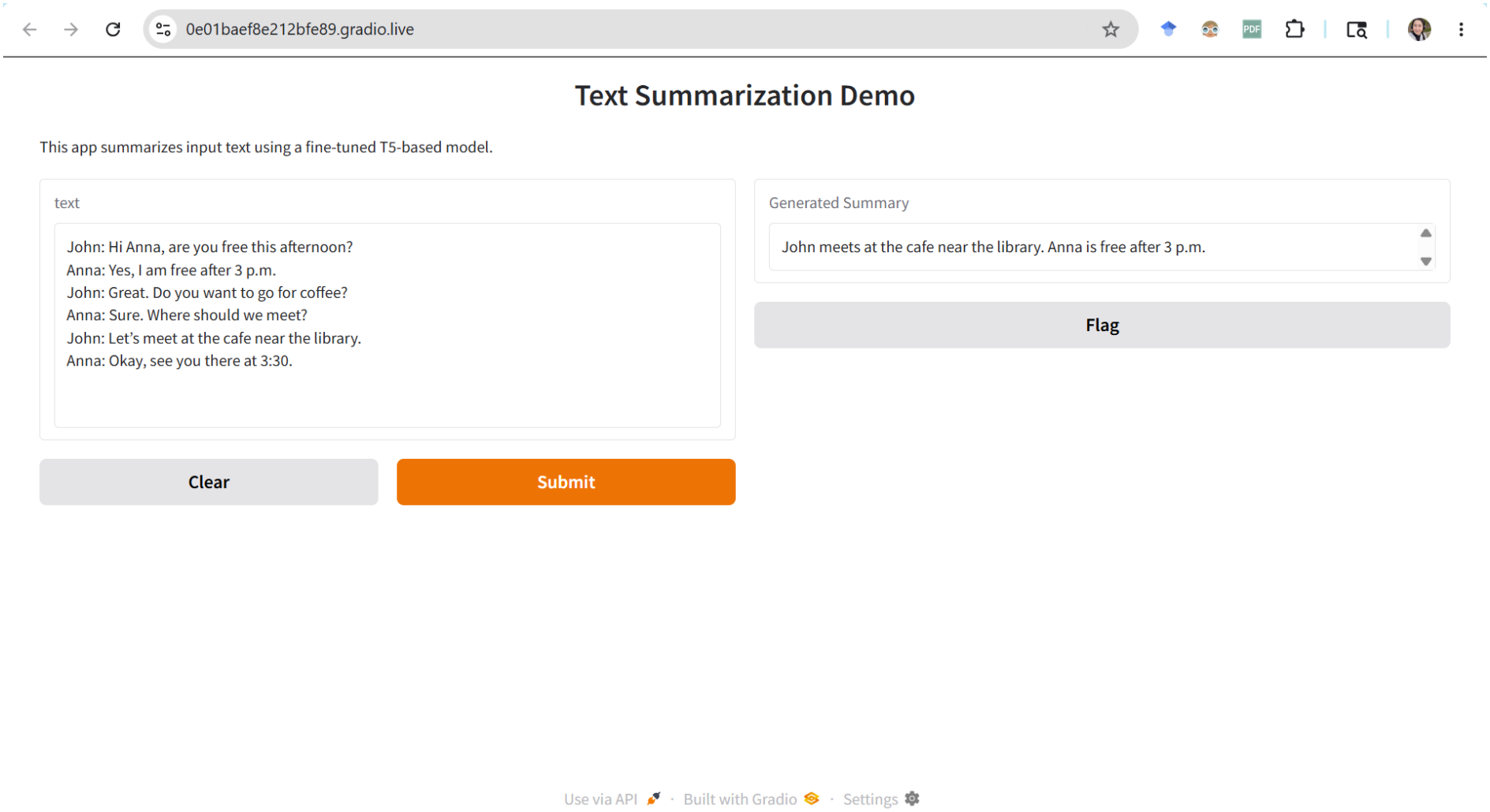
```
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://0e01baef8e212bfe89.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working director
```



Testing

Or, if you click on the provided URL, the one shown in blue, Gradio will open in a new browser tab:



Congratulations!

Now you know how to use Hugging Face models!

